601.433/633 Introduction to Algorithms **Fall 2026**
Homework #8 **Due:** November 12, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Boruvka's Algorithm (33 points)

Let $G = (V, E)$ be a connected, undirected graph and let $w : E \to \mathbb{R}_{\geq 0}$ be *distinct* edge weights (w is injective). We're going to analyze yet another MST algorithm: Boruvka's MST algorithm (from 1926), which is a bit like a distributed version of Kruskal.

We begin by having each vertex mark the lightest edge incident to it. (For instance, if the graph were a 4-cycle with edges of lengths 1, 3, 2, and 4 around the cycle, then two vertices would mark the "1" edge and the other two vertices will mark the "2" edge). This creates a forest F of marked edges. (Convince yourself that there won't be any cycles!). In the next step, each tree in F marks the shortest edge incident to it (the shortest edge having one endpoint in the tree and one endpoint not in the tree), creating a new forest F'. This process repeats until we have only one tree.

(a) (11 points) Show correctness of this algorithm by proving that the set of edges in the current forest is always contained in the MST.

We use a proof by induction via loop invariant on Boruvka's MST algorithm. We consider graph $G = (V, E)$ as described in the problem.

Base case. At the start, each $v \in V$ is a separate tree. Thus, there are 0 edges in the current forest. Since there are no edges in the current forest, every edge in the forest must be in the MST, proving the base case true.

Inductive step. We assume that for iterations 1 to $i - 1$ of Boruvka's, every edge added to forest F is in the MST for G. We prove that on iteration i, each edge in the new forest F' is in the MST for G.

We consider tree T in F. Boruvka's will select the lowest-weight edge $e = (u, v)$ such that $u \in T$ and $v \notin T$, meaning there will be no cycles. Therefore, we can consider edge e to be the minimum edge that crosses the cut $(T, \overline{T})$. We know, by the Inductive Hypothesis, that cut $(T, \overline{T})$ respects the MST. By **Theorem 17.2.6**, the lowest-weight edge e that crosses any cut respecting the MST is safe for the set of edges in a graph's MST. Thus, since e is the lowest-weight edge crossing $(T, \overline{T})$, then e is safe to add to the MST.

This process repeats for all trees in F. Since any cut respects the MST, any edge added between two trees in F is safe for the MST. Therefore, after iteration i of Boruvka's, every edge in the new F' is safe for G's MST.

In proving correctness for Boruvka's algorithm in the base and inductive case, we have proven correctness for Boruvka's algorithm as a whole.

(b) (11 points) Show how you can run each iteration of the algorithm in $O(m)$ time with just a few runs of DFS and no fancy data structures (heaps, union-find – remember, this algorithm was from 1926!). In other words, given a current forest F of marked edges, show how to find the set of edges which consists of the shortest edge incident to each tree in F in time $O(m)$.

For each iteration, we use DFS search to identify which vertices belong to which tree. We create a dictionary to keep track of, for each vertex v, which tree T_i within G such that $v \in T_i$. We run DFS on F, keeping track of which nodes are reachable from others. As per the notes in **Lecture 13**, we know that the time complexity of DFS is $O(n+m)$. However, we can simplify this further. Since G is a connected graph, we know that $m \geq n-1$, since the graph must contain at least enough edges to connect the n vertices. Rearranging, we get $m+1 \geq n$. Thus, $O(n) = O(m+1) = O(m)$. Inserting this into the DFS, we get that the time complexity of the DFS is $O(m)$.

From here, we initialize an array to keep track of the minimum outgoing edge from each tree T_i. In a graph with n vertices, there are at most n separate trees. Thus, this step takes $O(n) = O(m)$ time, as per the proof above.

Now that we know to which components each tree is in, we can now iterate through each edge in G. For each edge (u, v) in G, we check if $u \in T_i$ and $v \notin T_i$, and compare it against the current minimum outgoing edge weight from T_i. If (u, v) is smaller, it becomes the new minimum outgoing edge from T_i. Since this aspect of the iteration iterates through each edge in G once, we know it takes $O(m)$ time.

Once this round is done, we add each minimum edge to forest F. We add 1 edge for each T_i. Since this is a maximum of n trees, we know this takes $O(n) = O(m)$ time.

Summing this, we can see that each iteration of Boruvka's takes $O(m) + O(m) + O(m) + O(m) = O(m)$ time.

(c) (11 points) Prove an upper bound of $O(m \log n)$ on the running time of this algorithm.

The previous question showed that each iteration of takes $O(m)$ time. We will prove that there will be at most $\log n$ iterations of the algorithm, yielding overall time complexity $O(m) * \log n = O(m \log n)$.

We consider an arbitrary iteration of Boruvka's, where the forest F has k trees T in it. For each tree T_i, we select 1 outgoing edge from it to add to F. Since the edge is between 2 trees, adding it to the forest would join at least 2 edges. This means that every resulting tree in the new F' contains at least 2 of the trees in F. Therefore, the number of components k' is $k' \leq \frac{k}{2}$.

Thus, we know that each iteration at least halves the number of trees. We begin with n trees, so after i rounds, there are $k \leq \frac{n}{2^i}$ groups. Since the algorithm stops when the number of trees is 1, we know there must be 1 tree in F. We can calculate the number of rounds to reach 1 tree as:

$\frac{n}{2^i} = 1$

$n = 2^i$

$i = \log_2(n)$

Therefore, the number of iterations is $O(\log n)$. Since the amount of work per round is $O(m)$, we know the total time complexity is $O(\log n) \cdot O(m) = O(m \log n)$.

2 Clustering (33 points)

Consider the following problem. You are given a graph $G = (V, E)$ and a length function $\ell : E \to \mathbb{R}_{\geq 0}$. You should output a clustering (i.e., a partition) $(C_1, C_2, \ldots, C_k)$ of V into k clusters which maximizes the minimum distance between clusters. More formally, you should maximize $\min_{i,j \in [k], i \neq j} d(C_i, C_j)$, where $d(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$ and where $d(x, y)$ is the shortest-path distance (under length function ℓ). Design an algorithm for this problem which runs in $O(m \log n)$ time, and prove correctness and running time.

Hint: Think about MSTs and Kruskal's algorithm

TODO

3 Matroids (34 points)

(a) (17 points) Let U be a finite set and let $U_1, U_2, \ldots, U_k$ be a partition of U into nonempty disjoint subsets (where $k \geq 2$). Let $r_1, r_2, \ldots r_k$ be positive integers. Let $\mathcal{I} = \{S \subseteq U : |S \cap U_i| \leq r_i\}$ for all $i \in \{1, 2, \ldots k\}$. Prove that $(U, \mathcal{I})$ is a matroid.

TODO

(b) (17 points) Let $(U, \mathcal{I})$ be a matroid, and let $w : U \to \mathbb{R}^+$ be an assignment of weights to elements. We know that the greedy algorithm will return a maximum-weight independent set. But suppose we are also given some $I \in \mathcal{I}$, and want to return the maximum-weight independent set *that contains I*. Modify the greedy algorithm to solve this problem, and prove that your algorithm is correct. You do not need to analyze running time, but your algorithm should need only a polynomial amount of time and calls to the independence oracle.

TODO